Introdução à Computação
comparando arrays e aritmética de ponteiros

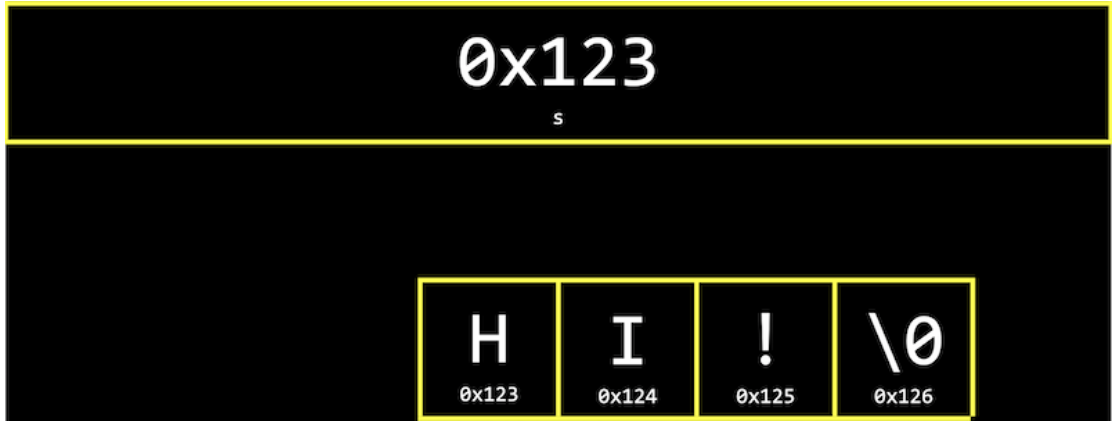
Alexandre Rademaker

Strings I

Como já vimos, strings são arrays de caracteres. Podemos declarar com

```
char s[] = "HI!".
```

Cada caractere, se está armazenado na memória, também tem um endereço único, e `s` é na verdade apenas um **ponteiro** com o endereço do primeiro caractere. Podemos acessar cada caracter como `s[0]`, `s[1]` etc.



Strings II

`s` é uma variável do tipo **char***, que é um ponteiro para um caractere.

Lembre-se de que podemos ler a string inteira começando no endereço em `s` e continuar lendo um caractere por vez da memória até chegarmos a `\0`.

Strings III

src/address02.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  typedef char* string;
5
6  int main(void) {
7
8    char s[] = "Hello";
9    char *t  = "Hello", *m = "World"; // it does not work
       with 'char* t = ..., m = ...'
10   string u = "Ola", v = "Mundo";    // why it works?!
11
12   int a   = 12; // declare a and put the value 12 on it
13   int *pa = &a; // pa is a pointer to int, put the address
       of a on it
14   *pa     = 24; // update the place pointed by pa to 24
15
16   /* error: incompatible integer to pointer conversion
       initializing
```

Strings IV

```
17      'int *' with an expression of type 'int' [-Wint-
    conversion]
18
19  int *pb = 24;  // declare a pointer to int but put a int
    value on it place.
20
21  */
22
23  printf("%s at %p %lu\n", s, s, strlen(s));
24  printf("%s at %p %lu\n", t, t, strlen(t));
25  printf("%s at %p %lu\n", m, m, strlen(m));
26  printf("%s at %p %lu\n", u, u, strlen(u));
27  printf("%s at %p %lu\n", v, v, strlen(v));
28 }
```

Strings V

Quando escrevemos `typedef char *string;` estamos criando um tipo de dados personalizado para a palavra string, tornando-a equivalente a `char *`.

Aritmética dos ponteiros I

Agora que sabemos o que são strings. Podemos imprimir um caractere por vez com

src/address03.c

```
1  #include <stdio.h>
2
3  int main(void) {
4
5    char *s = "HI!";
6    printf("%c  at %p\n", s[0], &s[0]);
7    printf("%c  at %p\n", s[1], &s[1]);
8    printf("%c  at %p\n", s[2], &s[2]);
9    printf("\\0 at %p\n", &s[3]);
10 }
```

Quando declaramos uma string com aspas duplas, o compilador decide como colocar esses caracteres na memória como um array.

Aritmética dos ponteiros II

Mas podemos usar os endereços diretamente

src/address04.c

```c
#include <stdio.h>

int main(void) {
  char *s = "HI!";
  printf("%c = %c = %c\n", *s, *(s + 0), s[0]);
  printf("%c = %c\n"      ,      *(s + 1), s[1]);
  printf("%c = %c\n"      ,      *(s + 2), s[2]);
}
```

`*s` lê o endereço armazenado em `s`, e `*(s + 1)` o endereço na memória 1 byte (char) maior.

`s[1]` é um **syntactic sugar**, uma abstração mais legível para humanos para a expressão `*(s + 1)`.

Aritmética dos ponteiros III

Pointer arithmetic é o processo de aplicar operadores matemáticos à ponteiros, tratando-os como números.

comparando arrays I

podemos comparar números

src/compare0.c

```c
#include <stdio.h>

int main(void) {

  int i = 12;
  int j = 45;

  printf("%s\n", i == j ? "same" : "different");
}
```

mas para comprar strings?

comparando arrays II

src/compare1.c

```c
#include <stdio.h>


int main(void) {

  char s[100];
  char t[100];

  printf("s: "); scanf("%s", s);
  printf("t: "); scanf("%s", t);

  // so compiler don't see s == t
  char *a = s;
  char *b = t;

  // always false!!!
  printf("%s\n", a == b ? "same" : "different");
  // only first char
  printf("%s\n", *a == *b ? "same" : "different");
}
```

Já vimos que para comparar strings precisamos da `strcmp`!

comparando arrays III

src/compare2.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main(void) {
5
6    char s[100];
7    char t[100];
8
9    printf("s: "); scanf("%s", s);
10   printf("t: "); scanf("%s", t);
11
12   // print strings
13   printf("%s at %p\n", s, s);
14   printf("%s at %p\n", t, t);
15   printf("%s\n", strcmp(s,t) == 0 ? "same" : "different");
16 }
```

comparando arrays IV

Cada string é um ponteiro para um endereço diferente? Não exatamente.

src/compare3.c

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main(void) {
5
6    char *s = "hello";
7    char *t = "hello";
8
9    printf("s = %p  t = %p\n", s, t);
10   printf("%s\n", strcmp(s,t) == 0 ? "same" : "different");
11 }
```

comparando arrays V

Será que podemos implementar nossa própria `strcmp`? Como seria?

1. recebe dois ponteiros para char e devolve um `int`
2. ler documentação e entender que a função deve retornar 0, algum positivo ou algum negativo.
3. um loop 'andando' pelos arrays recebidos até que algum `0` seja encontrado.

Veja `src/compare4.c`

comparando arrays VI

Em C, também podemos obter o endereço de `s` e `t` e armazenar em uma variável do tipo `char **`, um ponteiro para um ponteiro.

copiando arrays I

src/copy0.c

```c
#include <ctype.h>
#include <stdio.h>
#include <string.h>

int main(void) {
  char *s;
  char *t = s;

  printf("s:"); scanf("%s", s);

  // capitalize first letter in string
  if (strlen(s) > 0) {
    s[0] = toupper(s[0]);
  }

  printf("s: %s\n", s);
  printf("t: %s\n", t);
}
```

Ambas as saídas foram capitalizadas, por que?